
High-performance Computing

Exam, June 12, 2026, at 10:00 in P22

Those of you who received a bonus in the competition, mark the tasks for which you are claiming it.

1. Explain why UMA design does not scale beyond a small number of cores.

UMA (Uniform Memory Access) connects all cores to a single shared memory through a common bus. As cores are added, they contend for that one shared path, and total memory bandwidth stays fixed while demand grows. The bus saturates, contention and coherence traffic rise, and per-core effective bandwidth falls.

2. A parallel program divides work into units of highly variable duration. Explain why the OpenMP task pool mechanism achieves better performance than statically allocating these units to threads.

Static allocation maps work to threads in advance, with variable amount of work some threads finish early and wait for others. A task pool mechanism assigns next task to an idle, so work is dynamically balanced at runtime. This keeps all threads busy at the cost of some additional scheduling overhead.

3. The following code finds the maximum value in an array **a** of length *n* and stores it in **max_val** variable. Parallelise it efficiently using OpenMP pragmas.

```
int max_val = a[0];
#pragma omp parallel for reduction(max:max_val)
for (int i = 1; i < n; i++) {
    if (a[i] > max_val) {
        max_val = a[i];
    }
}
```

4. In what cases is it advantageous to use shared (local) memory in a CUDA kernel?

It is advantageous when data is reused by multiple threads in a block, since shared memory is much faster than global memory. Typical cases: data accessed repeatedly (stencil, tiled matrix multiply), thread communication within a block (synchronisation), staging global data to enable coalesced loads, and block-local accumulation (reduction, histogram).

High-performance Computing

Exam, June 12, 2026, at 10:00 in P22

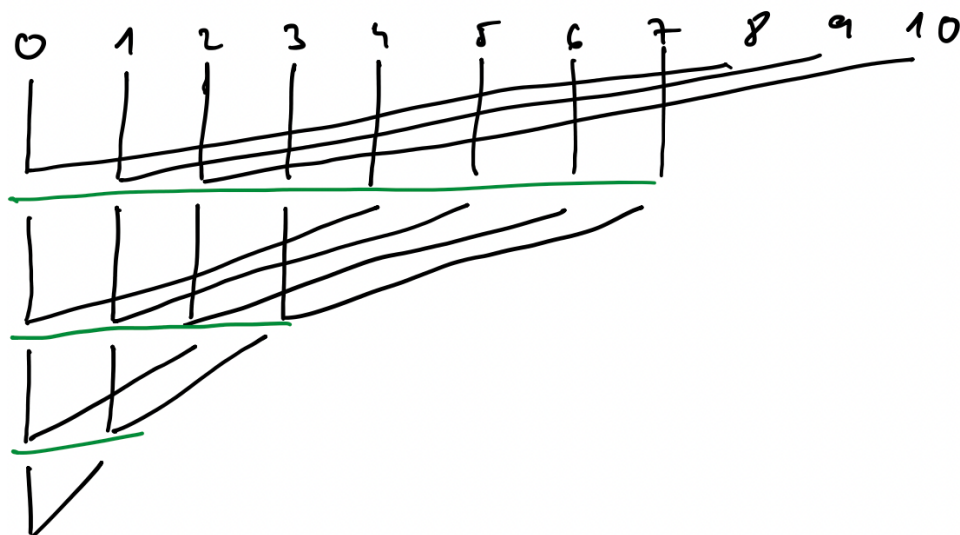
Those of you who received a bonus in the competition, mark the tasks for which you are claiming it.

5. The CUDA kernel below computes a histogram of the array `data`, whose `n` elements are integers from 0 to 9 only. The histogram `hist` has exactly 10 bins and is initialised to zero before the launch. There are at least as many GPU threads as there are elements in the array. Rewrite the kernel for better performance, using the fact that shared-memory operations are an order of magnitude faster than global-memory operations.

```
__global__ void histogram(int *data, int *hist, int n) {  
    int gid = blockIdx.x * blockDim.x + threadIdx.x;  
    if (gid < n) {  
        int bin = data[gid];  
        atomicAdd(&hist[bin], 1);  
    }  
}
```

```
__global__ void histogram(int *data, int *hist, int n) {  
  
    __shared__ int local_hist[10];  
    if (threadIdx.x < 10)  
        local_hist[threadIdx.x] = 0;  
    __syncthreads();  
  
    int gid = blockIdx.x * blockDim.x + threadIdx.x;  
    if (gid < n) {  
        int bin = data[gid];  
        atomicAdd(&local_hist[bin], 1);  
    }  
    __syncthreads();  
  
    if (threadIdx.x < 10)  
        atomicAdd(&hist[threadIdx.x], local_hist[threadIdx.x]);  
}
```

6. Sketch a parallel reduction scheme that sums 11 elements in the least number of steps. The scheme should be optimized for GPU kernel programming.



High-performance Computing

Exam, June 12, 2026, at 10:00 in P22

Those of you who received a bonus in the competition, mark the tasks for which you are claiming it.

7. HPC clusters are often built with two separate networks — typically Ethernet and InfiniBand. Explain the distinct role each network serves and the key characteristics (e.g. latency, bandwidth, cost) that make each suited to its role.

InfiniBand is the high-performance interconnect with very low latency, very high bandwidth, being ideal for tightly coupled MPI. Ethernet with higher latency and lower bandwidth serves management, provisioning, login, monitoring, and access to storage. Separating them keeps latency-critical MPI traffic off the general-purpose network.

8. Two MPI processes with ranks 0 and 1 exchange an array of length n , each sending from buffer `snd` and receiving into buffer `rcv`, as shown below.

```
MPI_Send(snd, n, MPI_INT, (rank+1)%2, 0, MPI_COMM_WORLD);  
MPI_Recv(rcv, n, MPI_INT, (rank+1)%2, 0, MPI_COMM_WORLD, &status);
```

Explain why this code may deadlock.

MPI_Send is blocking but behaviour depends on the data size. For small n it uses MPI_Bsend which sends data to the MPI buffer and continues to MPI_Recv without any deadlock. For large n it uses MPI_Ssend which waits for the matching receive to take data. In the latter case the code deadlocks as both processes wait for partner to respond.

9. In image processing, edge detection is commonly performed by convolving an image with a small filter like a 3×3 Sobel kernel, where each output pixel is computed from the corresponding input pixel and its neighbours. Which parallel computation pattern does this correspond to? Briefly justify your answer.

It is a stencil pattern as each output pixel is a fixed-shape function of an input pixel and its neighborhood, and a pixel computation is independent of other pixel computations.

10. A parallel program on 16 processors achieves a speedup of 10. Using Amdahl's law, calculate the maximum achievable speedup of the program.

$$\text{Amdahl's law: } S = \frac{1}{f + \frac{1-f}{p}} \longrightarrow f = \frac{\frac{1}{S} - \frac{1}{p}}{1 - \frac{1}{p}} = \frac{1}{25}$$

$$\text{Limit } p \rightarrow \infty: S = \frac{1}{f} = 25.$$
